ASSIGNMENT 2
GROUP 14

ANUSHKA GUPTA	**220101014**
KASHISH AGGARWAL	**2201010150**
MAHEK CHOUDHARY	**220101062**
MAHUA SINGH	**220101063**

HALF ADDER :

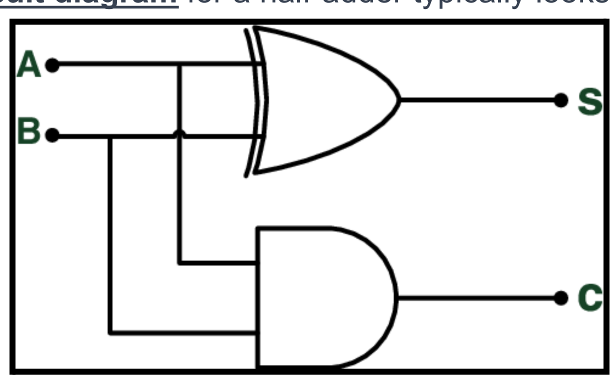
- A half adder is a digital circuit that performs binary addition on two input bits, producing a sum bit (S) and a carry bit (C). It's called a "half" adder because it can only handle two inputs and doesn't consider any carry from previous additions.

- The half adder has two input bits, typically labeled as A and B.

The truth table for a half adder is as follows:

A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

- Here's how a half-adder works:
  - XOR (Exclusive OR) Gate (Sum Bit - S):
    - The XOR gate takes the two input bits (A and B) as its inputs.
    - The output of the XOR gate is the sum bit (S).
    - The XOR operation results in a 1 only when the inputs are different.
  - AND Gate (Carry Bit - C):
    - The AND gate takes the same input bits (A and B) as its inputs.
    - The output of the AND gate is the carry bit (C).
    - The AND operation results in a 1 only when both inputs are 1.
  - So, the sum bit (S) represents the bitwise addition of A and B, and the carry bit (C) represents the carry generated when adding A and B.
    - The **circuit diagram** for a half-adder typically looks like this:



FULL ADDER:

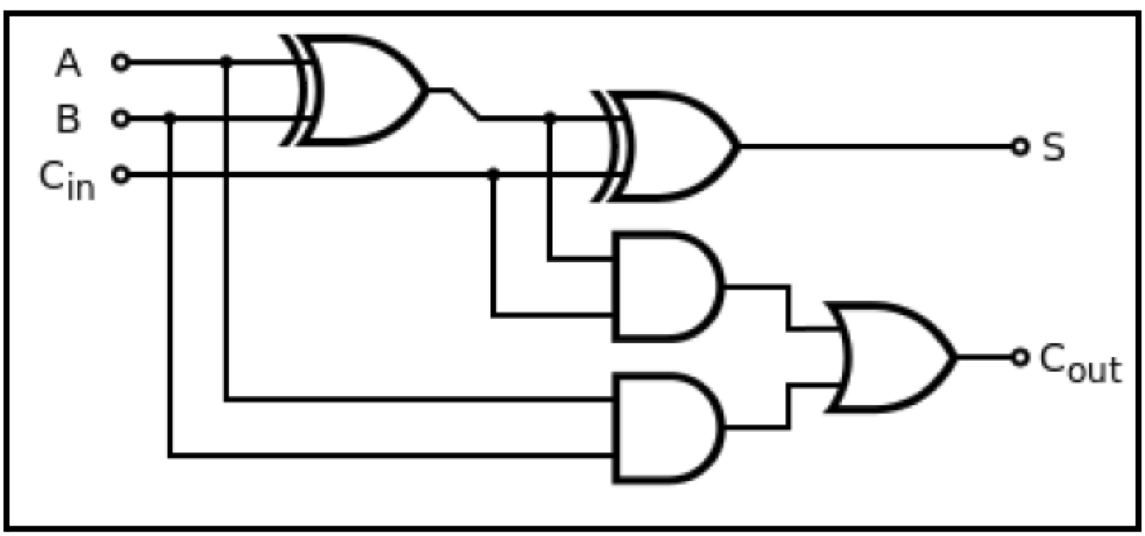
1. While a half adder can add two binary digits, a full adder can add three binary digits along with a carry input from the previous stage. It computes the sum of three input bits (input_A, input_B, and input_carry_in) and produces two outputs: Sum (S) and Carry-out (output_carry_out).

TRUTH TABLE FOR FULL ADDER

The truth table for a full adder is as follows:

A	B	Cin	Sum (S)	Carry-out (Cout)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

GATE LEVEL FOR FULL ADDER



2. Here's how the logic works:
   The Sum(output_sum) results from the XOR operation between the three input bits (A, B, and input_carry_in). It's 1 if the number of 1s among the inputs is odd.

The Carry-out (output_carry_out) is produced by two AND gates and one OR gate.

- The first AND gate computes the carry between A and B (input_A AND input_B),
- The second AND gate computes the carry from the previous stage and the current inputs ((input_A XOR input_B) AND input_carry_in).
- The OR gate combines these two carry signals.

The full adder allows for adding multiple bits by propagating the carry from one stage to the next.

RIPPLE CARRY ADDER :

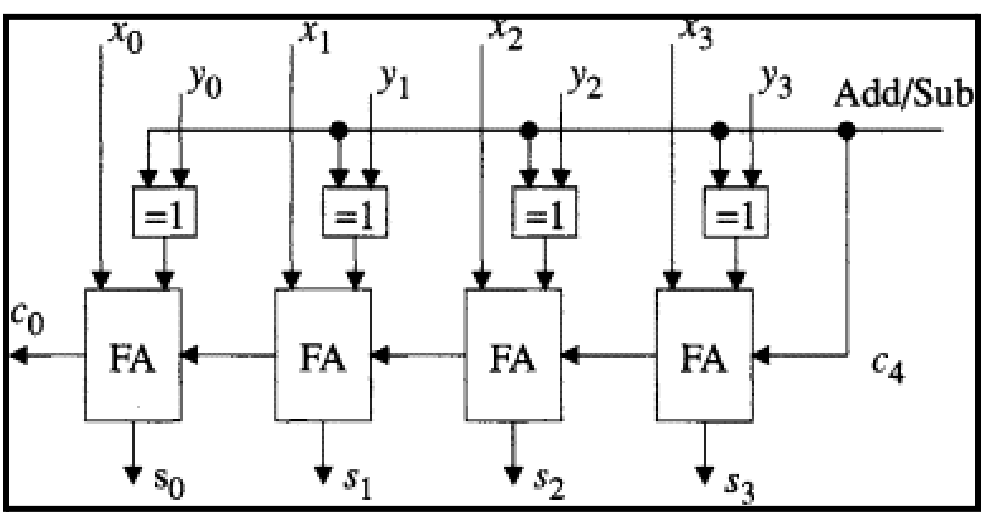
- A ripple carry adder is a digital circuit used for the addition of multiple binary numbers. It's called a "ripple carry" adder because the carry bit ripples through the successive stages of the adder. This type of adder is composed of multiple full adders connected in series.

- Let's understand the working of a 4-bit ripple carry adder as an example. In this case, four full adders are connected in series, with each full adder handling a pair of bits from the input numbers and the carry from the previous stage.

- Here's how it works:
  - Each full adder (FA1, FA2, FA3, FA4) takes two input bits (Ai, Bi) and a carry bit (Ci) from the previous stage.
  - The sum bit (Si) and the carry-out bit (Ci+1) are produced by each full adder.
  - The carry bit from the first stage (Cin) is set to 0 initially.
  - The sum bits (S0 to S3) from each full adder form the final binary sum, and the carry-out bit (Cout) from the last stage is the carry for the next higher bit, if needed.

- The key idea is that the carry ripples through the stages, and each full adder computes its sum and carry based on the inputs and the carry from the previous stage. The downside of ripple carry adders is that the carry propagation introduces a delay, making them slower than other adder designs like carry-lookahead adders or carry-skip adders.

CIRCUIT DIAGRAM



TRUTH TABLE

A1	A2	A3	A4	B4	B3	B2	B1	S4	S3	S2	S1	Carry
0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	1	0	0	1	0	0	0	0
1	0	0	0	1	0	0	0	0	0	0	0	1
1	0	1	0	1	0	1	0	0	1	0	0	1
1	1	0	0	1	1	0	0	1	0	0	0	1
1	1	1	0	1	1	1	0	1	1	0	0	1
1	1	1	1	1	1	1	1	1	1	1	0	1

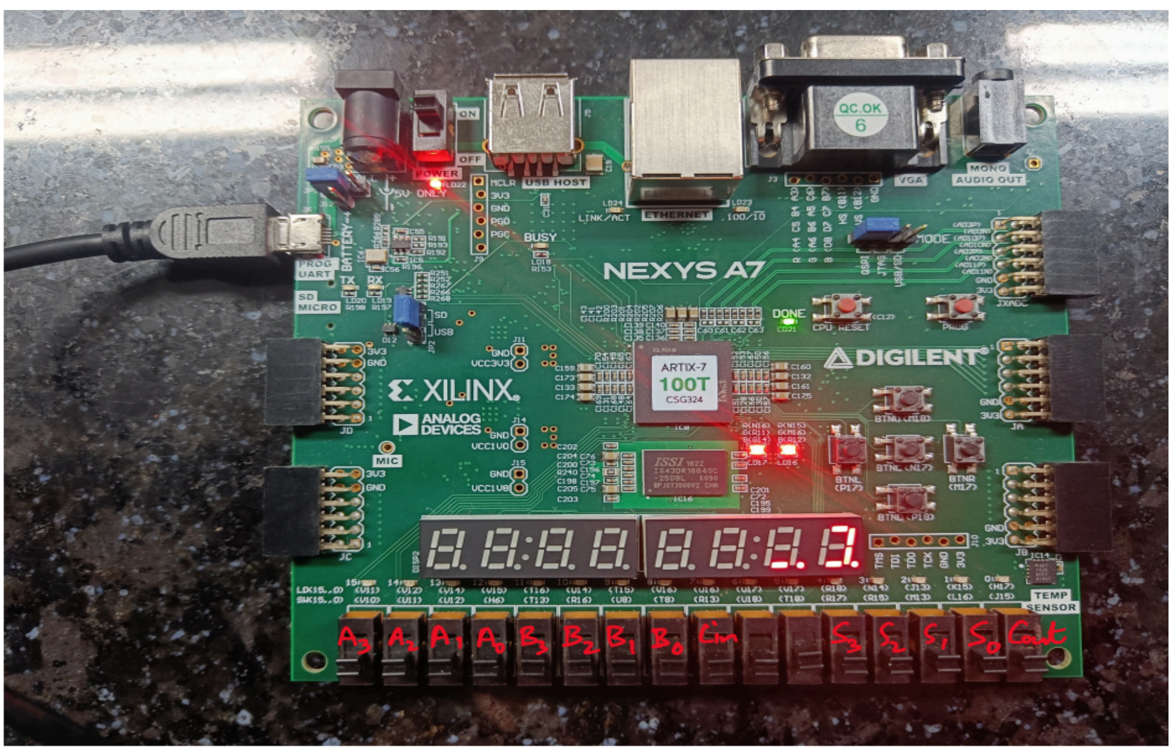
FPGA :

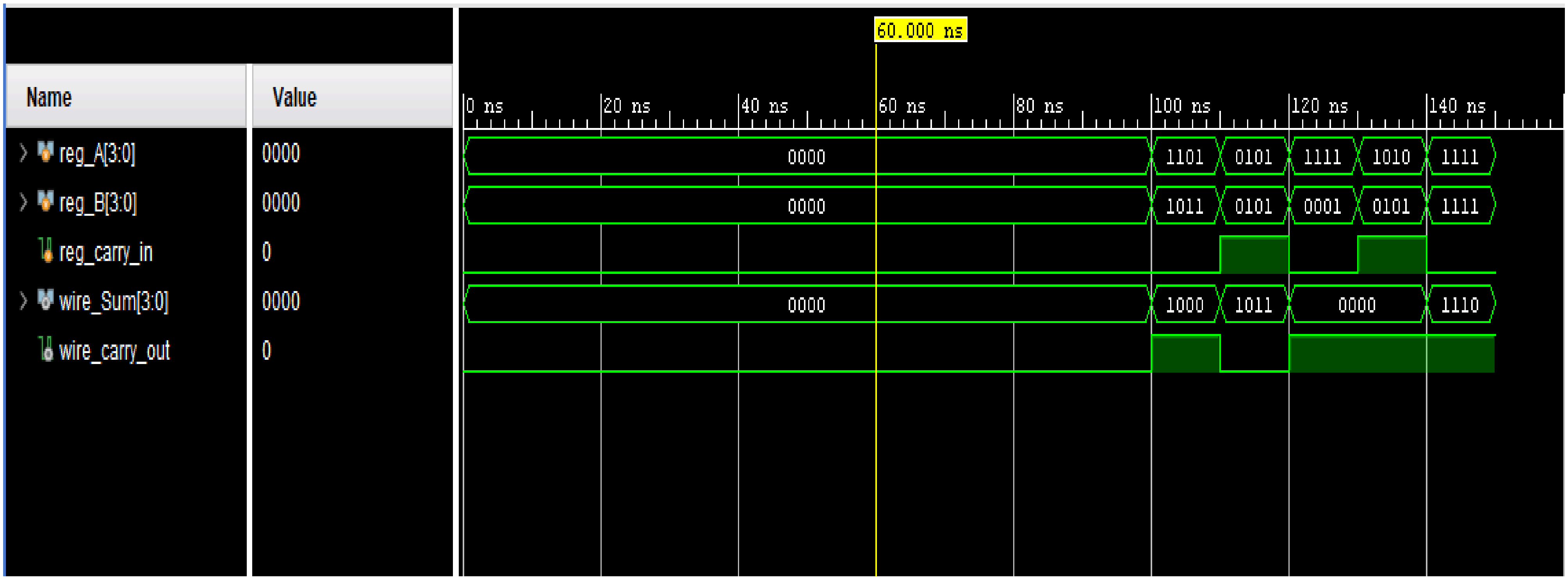
- FPGA stands for Field-Programmable Gate Array. It is a type of programmable logic device that offers flexibility in digital circuit design and implementation.

- Unlike fixed-function integrated circuits (ICs) such as microprocessors and microcontrollers, FPGAs can be reprogrammed and configured by the user for a wide range of applications.

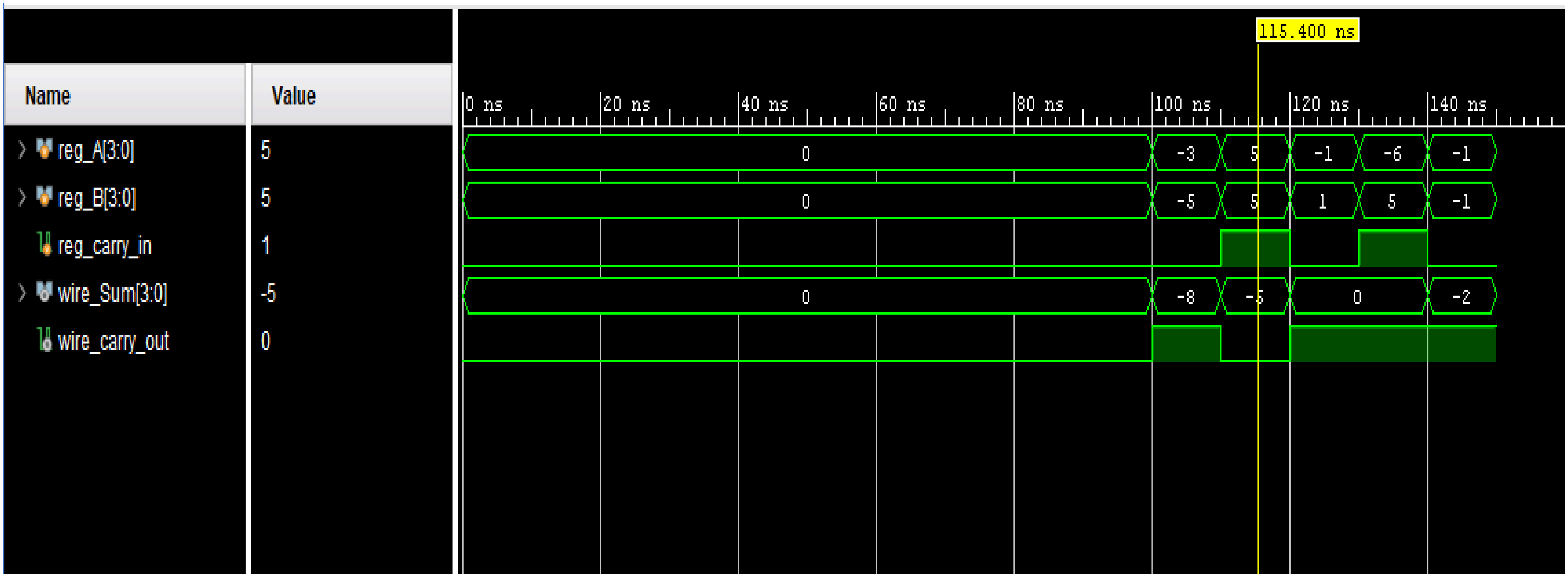
- Here are some key features and aspects of FPGAs:
  - **Configurability**: Programmable for diverse digital circuit implementations.
  - **Logic Blocks**: Configurable logic units for implementing functions and storing data.
  - **Interconnects**: Programmable connections between logic blocks.
  - **Reprogrammability**: Can be reconfigured multiple times for flexibility.
  - **Parallelism**: Excels in simultaneous processing of multiple operations.
  - **Applications**: Widely used in signal processing, telecommunications, etc.
  - **Development Tools**: Require hardware description languages and manufacturer-provided tools.

OUR SELECTED PORTS (FPGA)
(GROUP 14)





Name
Value
reg_A[3:0]
0000
reg_B[3:0]
0000
reg_carry_in
0
wire_Sum[3:0]
0000
wire_carry_out
0
60.000 ns
0 ns
20 ns
40 ns
60 ns
80 ns
100 ns
120 ns
140 ns
0000
1101
0101
1111
1010
1111
0000
1011
0101
0001
0101
1111
0000
1000
1011
0000
1110


Name
Value
reg_A[3:0]
5
reg_B[3:0]
5
reg_carry_in
1
wire_Sum[3:0]
-5
wire_carry_out
0
115.400 ns
0 ns
20 ns
40 ns
60 ns
80 ns
100 ns
120 ns
140 ns
0
-3
5
-1
-6
-1
0
-5
5
1
5
-1
0
-8
-5
0
-2